Chronicle
Video-free memory (VFM) architecture

Technical note — 27 October 2026

Branch `experiment/video-free-memory` — main UI + VLM-first answers

One-sentence summary. Chronicle compiles a video episode *once* into a compact, timestamped memory — VLM captions, a salience-coalesced event ledger, a few WebP evidence frames and an LLM-ready pack — then answers questions from that memory in microseconds, with no video and no video model in the query path. The zine dashboard from `main` is the UI; the VLM memory is the answer engine.

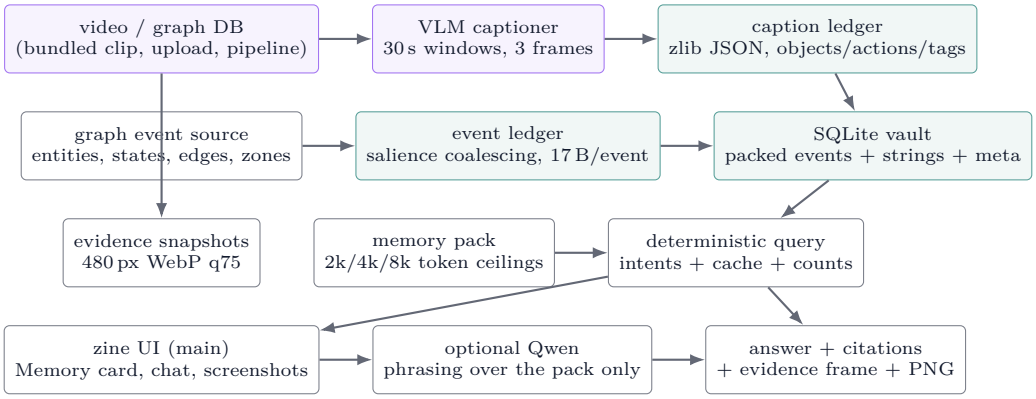
1 What is different from the graph-only dashboard

The `main` dashboard replays a live graph: entities, state transitions, spatial edges and zones. That is excellent for visual inspection, but it has two limitations for question answering:

- It only knows what the graph recorded. Replay clips often have **state/edge events but no VLM interaction table**, so "what happened around 10 s?" had no semantic answer.
- Labels can arrive late. If an entity row is emitted at `last_seen`, the first half of the episode shows anonymous `Track` rows.

VFM adds the layer the graph does not keep: **dense natural-language captions** at 30 s resolution, a compact event ledger derived from the graph, evidence snapshots, and a retrieval engine whose answers are deterministic. The graph remains the index; the captions and events are the recallable memory.

2 Architecture at a glance



The two inputs are compiled in parallel and joined in the vault. Query time reads only the vault and the pack. The source video is retained for playback and screenshot capture, but is never required to answer a question.

3 Glossary: what the terms mean

Term	Meaning in Chronicle
VFM	**Video-Free Memory**. The compiled artefact that answers questions without the source video: packed events, captions, evidence references and metadata.
VLM	Vision-language model. Used *once at compile time* to caption frames/windows (default `qwen2.5vl:3b`); not used for deterministic answers.
Graph DB	The live per-episode SQLite graph: entities, states, edges, zones, anchors and observations. Source of truth for identity, state and counts.
Vault	The compact SQLite file written by the compiler: struct-of-arrays events, string dictionary, captions, evidence index and metadata.
Caption ledger	zlib-compressed JSON of 30 s (or 20 s) windows. Each window has a caption, objects, actions, zone, tags and 3 timestamped frame captions.
Event ledger	Salience-coalesced state/edge/interaction events from the graph. Packed at about 17 bytes/event.
Salience coalescing	Filtering visibility noise (camera pan, occlusion, reacquisition) and merging repeated touches/placements into memory-worthy events.
Evidence still	A 480 px WebP snapshot selected at an interaction, crop, zone keyframe or anchor. Stored with a timestamp and relative path.
Memory pack	LLM-ready Markdown/JSON: verified answer + ranked captions/events + evidence references + answering rules. Budgeted at 2k/4k/8k tokens.
Deterministic query	Intent + time + entity lookup over the vault. No LLM, no video, no frame sampling; answers carry citations.
Track / peak / physical	Track = raw tracker id count (upper bound); peak = most same-label tracks visible in one frame (lower bound); physical = tracks after cross-label duplicate and transient removal (estimate).
Auto-compile	Building the vault once on first view of a clip, in the background. Query stays on the graph fallback until it is ready.
Frame chip	The per-answer `frame @ t` control: seek to the cited time, capture that frame, open it in a save/fullscreen lightbox.
Graph fallback	If VFM has no answer, the same chat asks the live graph engine and labels the answer `graph-fallback`.

4 End-to-end workflow

A. Graph-backed clip (bundled / ingested run)

1. **Ingest.** Perception writes entities, state transitions, edges, anchors, zones and observations into an episode DB. The graph state machine emits a transition per meaningful change.
2. **Verify important actions.** Interaction clips are queued; a temporal strip (pre/contact/post) is built and the action verifier (Florence-2 path) returns an action, confidence and rejection reason. Low-confidence/rejected clips stay in the review queue.
3. **Compile (on first view or via the button).** The service snapshots the DB, captions the video once, builds the salience event ledger, writes the vault, extracts evidence stills, renders `memory.html`, writes the memory pack and zips the downloads.
4. **Answer.** The zine chat routes to the vault. A question resolves an intent (where / when / around / before / after / count / holding / inventory / yes-no), looks up captions/events/time and returns an answer with citations and an evidence reference.
5. **Inspect.** The answer shows citation chips, a `frame @ t` button, an evidence thumbnail only when the still is within 8 s of the question, and the graph fallback if the vault has no match.
6. **Export / import.** Export writes one ZIP with the full current timeline, snapshot, CSVs, all replay timelines and the VFM artefacts; import reloads a timeline without re-ingesting video.

B. Caption-first clip (fresh upload / long street walk, no DB)

1. **Upload / select** the video. No perception DB is required.
2. **Caption once:** 30 s windows (20 s for short clips), 3 frames each. A window caption and per-frame captions become the ledger; objects/actions become coarse entities and `SEEN` events.
3. **Build the caption vault, evidence and pack** with the same compiler stages.
4. **Answer** from the caption/event ledger. Counts are caption coverage, not physical counts; the UI says so.
5. **Inspect / export** exactly as in the graph-backed path.

Episode	Duration	Compile (once)	Deterministic query p50
Synthetic demo	12 s	∼12 s	0.05–0.06 ms
Aria walk	60 s	∼20 s	0.05–0.06 ms
London walk	15 min	∼5 min	0.89 ms (cold), 3 ms cache

5 What is saved now (more than the graph)

The graph is still saved. VFM adds four new artefacts. All of them are derived once, compressed, and stored next to the source run; the video itself is optional at query time.

Artefact	**What it contains**	**Demo**	**Walking**
Graph DB	entities, states, edges, zones, observations (source of truth)	168 KB	254 KB
Packed vault	struct-of-arrays events, string dictionary, metadata, answer cache	28.7 KB	31.7 KB
Caption ledger	30 s VLM windows: caption, objects, actions, zone, tags, 3 frame captions	1 window	2 windows
Event ledger	salience-coalesced state/edge/interaction events	8 events	11 events
Evidence	480 px WebP stills, ranked by interaction / crop / zone keyframe	8 frames	8 frames
Memory pack	verified answer + ranked captions/events + evidence refs + rules	2.4 KB	3.0 KB
Report	single-file offline HTML with timeline, captions, query box	366 KB	410 KB
Whole job dir	vault + ledger + evidence + report + pack + zip	**988 KB**	**1.3 MB**

The report and zip duplicate bytes for convenience; the *at-rest memory* is the vault + ledger + evidence. On a 60-minute synthetic episode (2,474 events, 184 caption windows, 40 snapshots) the vault is 142 KB and the whole memory stays far below the source video.

5.1 Why this is more information

- **Semantics.** The graph knows `person VISIBLE_MOVING`; the caption knows "a person walks on a sidewalk holding a cup of coffee". The second is what a user asks for.
- **Coverage.** Captions cover the whole episode, including segments where the tracker emitted no interaction row.
- **Recall by time.** Caption windows are timestamped, so "around 10 s" is a range query, not a similarity search over one frame.
- **Evidence.** Answers point to stored stills, so the UI can show the frame and seek the video without re-ingesting.
- **Counts.** Compile-time observations also yield track counts, peak simultaneous visibility, and cross-label duplicate merges (Section 8).

5.2 Design lineage

The design deliberately borrows one idea from each line of work: caption-first episodic QA [4], time as a first-class key [5], query-agnostic high-recall memory [6], egocentric entity/event schemas [3, 2, 1], graph retrieval [11], tiered sensory/short/long-term memory [10], event segmentation [9], bounded retention [8], text-first / frames-on-demand [7, 16], semantic answer caching [13], and personal-memory retrieval [12]. The model-native memory work (Titans, MemoryLLM) [14, 15] is a north star, not a dependency.

6 Compile pipeline

1. **Snapshot the source DB.** SQLite's backup API makes a consistent copy while the ingest worker may still be writing.
2. **Caption once.** Split the video into 30 s windows; sample three frames per window; caption each frame with a VLM (`qwen2.5vl:3b` by default) and synthesise a window caption. Store zlib-compressed JSON with objects, actions, zone and tags.
3. **Build the event ledger.** Read entities, state transitions, anchors and edges; coalesce visibility noise and repeated touches into salient events. Pack them struct-of-arrays at roughly 17 bytes/event.
4. **Build the vault.** Write 1 KB SQLite pages, the string dictionary, packed events, captions and metadata (including per-label count summaries).
5. **Extract evidence.** Save a bounded set of 480 px WebP q75 stills (interaction strips, object crops, zone keyframes). No full-resolution frames are retained.

6. **Render the report.** Write a single-file HTML with the timeline, captions and a client-side query box; embed evidence as data URLs.
7. **Write the pack.** Produce an LLM-ready Markdown/JSON pack. Budgets of 2k/4k/8k tokens keep the verified answer and rules, then add windows/events until the ceiling.
8. **Zip and manifest.** Expose `vault.db`, `memory.html`, `ledger.json`, `memory_pack.md/json`, CSVs and `evidence/`.

The compile is the only expensive step (seconds to tens of seconds, dominated by VLM calls). In the UI it runs *once per clip, in the background*, on first view. After that, opening the clip or asking a question does not touch the video.

7 Query path and latency

Deterministic vault. The query engine resolves an intent (where/when/around/before/after/ count/inventory/yes-no/keyword), looks up entities and events by time, and returns an answer with citations and an evidence reference. A canonical answer cache makes paraphrases hit the same entry. No LLM, no video, no frame extraction in this path.

Qwen (optional). The pack is passed to a local text model for phrasing (`qwen2.5:7b-instruct`). The model never sees frames; it sees the verified structured answer, timestamped captions, events and evidence references. It is an optional polish layer, not a dependency.

Path	Accuracy (30 questions)	p50	Video at answer time
Deterministic VFM vault	**30/30**	**0.05–0.06 ms**	no
Qwen 7B from memory pack	**30/30**	1.07 s	no
Original graph + Qwen 7B	14/30	0.85 s	yes
9-frame contact sheet + Qwen2.5-VL	7/30	4.46 s	yes (19.24 MB)

The 30-question set is an internal regression suite, not a public benchmark. It is useful because it covers location, temporal, count, negation, before/after and paraphrase questions on the same episodes.

8 Counts: tracks, visibility and duplicates

Count questions need more than one number. At compile time we compute, per label:

Field	Meaning
`tracks`	raw number of tracker ids with that label (upper bound)
`peak`	max same-label tracks present in one observation frame (lower bound)
`physical`	tracks after removing cross-label duplicate detections and one-frame transients
`duplicates`	tracks that were near-identical to a track under another label
`merged_into`	which label absorbed those duplicates

A duplicate is accepted only when two different-label boxes share the same frame with IoU ≥ 0.90 and centre distance ≤ 5 px, and one label has more total observations. This is deliberately conservative: same-label co-visible boxes on cook have max IoU ≈ 0.01, so they are *not* merged.

On the cook clip this turns four "bowl" tracks into **zero physical bowls**: two of the plates were emitted under both `bowl` and `plate` (IoU 0.91–0.99), and a fourth track was a one-frame detection. The plates resolve to two physical plates. On walking, two car tracks are one object at a time, so the answer is a range (1–2), not a false single count.

9 Dashboard integration

The zine mission control from `main` is the default UI (`./chronicle.sh start`, port 8080 in main, 8082 in this worktree). VFM is added behind it, not as a second uploader:

- **Auto-compile.** On first view of a clip with a graph DB, the vault builds in the background. The Memory card shows progress, then report and downloads.
- **VLM-first chat.** Once ready, the existing chat routes to the vault (`provider: vfm-memory`) and answers in microseconds. If not ready, a state/edge fallback still answers "around t" instead of claiming no interactions exist.

- **Screenshot at answer time.** Every grounded answer gets a frame chip that seeks to the citation, captures the exact frame, stamps clip + timestamp, and opens a lightbox.
- **Fullscreen.** Stage toolbar `S` (screenshot) and `F` (fullscreen); the screenshot lightbox has its own fullscreen button.
- **Downloads.** `vault.db`, `memory.html`, `memory_pack.md`, `memory_pack.json`, `memory.zip`.

10 Event-driven evidence and verification

The trigger for "something important happened" is the **graph state machine**, not the captioner. It emits transitions such as `NEW`, `INTERACTED`, `HELD`, `CONTAINED`, `PLACED_ON`, `LOST`. Those transitions drive two bounded systems:

- **Verification queue.** Interaction clips are de-duplicated, bounded and queued. A temporal strip gives the verifier pre/contact/post frames; the Florence-2 path returns an action primitive, confidence and a rejection reason. Rejected or ambiguous actions are inspectable in the review queue/telemetry.
- **Evidence selection.** At compile time, `extract_evidence.select_events` picks major markers (`INTERACTED`, `HELD`, `CONTAINED`, `LOST`, `NEW`, `PICK`, `PLACE`, `INSERT`, anchors/keyframes), enforces a 2 s minimum gap and a 40-still cap, then writes 480 px WebP stills with an append-only index.

The standalone `evidence_recorder.py` prototype can write 640 px JPEGs plus a JSONL index. The zine pipeline now also includes an **event-clip recorder**: transition, low-confidence and label-conflict triggers receive a readable salience score, coalesce within 2.5 s, and become a 3 s pre + 3 s post `clip.mp4`, a rebased `events.jsonl` replay and a `graph.json` snapshot (entity/zone/edge nodes at the anchor time). It then extracts **three frames** (1 s pre / contact / 1 s post), sends them to Qwen2.5-VL (or Florence-2 as fallback), appends the verified interaction to the timeline and compiles a small queryable per-event `vault.db`; one `snapshot.jpg` is retained for audit. A disagreeing VLM answer never overwrites a strong graph state: the disagreement is flagged for review. The **events** button and `POST /api/event-clips` expose the whole bounded path; see `docs/EVENT_CLIPS.md`. There is still **no learned live anomaly score** and the VLM is not called on every strange moment. That remains a deliberate scope choice: the recorder is bounded, the trigger is auditable, and only selected clips pay the VLM cost.

The bounded shape is: transition / low confidence / label conflict → salience score → 3 s pre + 3 s post clip → 3 frames to Qwen2.5-VL / Florence-2 on that clip only → verified interaction + snapshot + per-event vault. The event-clip recorder implements this path (`POST /api/event-clips`); the VLM pass is optional when no model is reachable and falls back to the graph hint plus the rule classifier. This keeps the cost bounded and makes the trigger auditable.

11 Is this a good system?

Strengths

- **Speed and cost**: answers come from a 95 KB vault, not a 249 MB video; cold p50 is sub-millisecond to a few milliseconds and repeat answers are cached.
- **Coverage**: captions cover the whole episode, including segments where the tracker emitted no interaction row.
- **Video-free**: after compile, query, report, pack and evidence do not need the video. The source is kept only for playback/screenshots.
- **Evidence and audit**: timestamped citations, stored stills, frame chips, fullscreen and ZIP export.
- **Deterministic**: no model in the default answer path, so it is reproducible and cheap on stage.
- **Composable**: the graph is still the source of identity/state/counts and the fallback when captions have no answer.

Where it is weaker

- **Lossy compression**: captions are a summary; fine-grained motion, small text, audio and subtle social cues are lost.
- **Caption hallucination**: the VLM can misdescribe an object. Evidence and citations make it auditable, not infallible.
- **Counts**: track/peak/physical are estimates, not hand-labelled ground truth for every episode.
- **Long video without a DB**: no graph, so no state/edge fallback and counts are caption coverage only.
- **Internal evaluation**: the 30-question suite is a regression test, not a public benchmark.

Verdict: it is a *good retrieval-and-recall system*, not a replacement for a full video-reasoning model. It is strongest when the question is "what happened / where / when / how many / what is X holding", and weakest on "why / exactly how / what did the tiny object do".

12 How this compares with common industry approaches

Approach	Answer path	Strength	Weakness
Frame-sampling video LLM	Sample N frames per question and send them to a VLM	Open-ended reasoning, no compile step	Re-samples every question; cost/latency scale with length; coverage depends on sampling; video must be available
Long-video RAG / frame retrieval	Embed frames/clips, retrieve top-k per question, then VLM	Better coverage than fixed sampling; standard tooling	Retrieval misses; still pays a VLM per query; frames remain the substrate
Hierarchical memory / caption QA	Compile captions/events once, retrieve text, optionally verify	Cheap reuse, long-horizon coverage, no video at query	Lossy captions; retrieval/segmentation design; less fine-grained than pixels
Chronicle VFM (this work)	Compile captions + event ledger + evidence once; deterministic lookup; optional text LLM	**Sub-ms to ms answers; 249 MB → 95 KB; evidence-backed; offline; graph fallback**	Caption loss; internal benchmark; no audio; not a general video-reasoning model

So: **not categorically "better than industry"**, but a better *cost/latency/coverage trade-off for egocentric recall and demo QA*. Industry systems optimise open-world reasoning; this optimises repeated, evidence-backed questions over a long episode. The right claim is "complementary", not "replaces a video LLM".

13 Pitch: what to say and show

One line

"We turn a 15-minute, 249 MB video into a 95 KB searchable memory; questions are answered in under a millisecond, with no video and with evidence."

30 seconds

1. Everyone else re-watches or re-samples the video for every question. That is slow and expensive.
2. We compile once: VLM captions, a salience event ledger, a few evidence stills and a small memory pack.
3. After that, the vault answers deterministically in under a millisecond; an LLM is optional for phrasing only.
4. The same UI keeps the graph, playhead and evidence, and exports/imports the whole memory as a ZIP.

Demo beats

1. Play the 15-minute walk; show the video size and the 95 KB vault.
2. Ask "what happened at 10 minutes?" and "what is the person holding?"; show citations and the frame chip.
3. Ask a count question; show the track/peak/physical estimate and the deduped cook bowls example.
4. Screenshot the cited frame, open fullscreen, export the graph ZIP, import it back.
5. Switch to the graph fallback and show an exact state/entity query.

Answers to judge questions

- **"Isn't this just captions?"** No: captions + a salience event ledger + evidence + count reconciliation + a deterministic intent engine. Captions alone cannot answer "when", "how many" or "where" as cleanly.
- **"Why not just use a video LLM?"** We do, at compile time. Query time is where the cost/latency wins; a per-question video model is the baseline we measured against (14/30 graph+Qwen, 7/30 9-frame VLM).
- **"What is lost?"** Fine motion, audio, tiny text and some rare objects. We state that plainly and keep evidence for audit.
- **"Is 30/30 a benchmark?"** No, it is an internal regression suite. CapMem/EgoSchema/Ego4D would be the external next step.

14 Honest limits

- `physical` is a conservative estimate, not hand-labelled ground truth for every episode. Cook plates/bowls were checked visually; walking cars were not.
- Counts are label-dependent. An armchair labelled `sofa`, or a bowl never co-detected with a plate, still contributes to the wrong label.
- Caption-first uploads (no graph DB) have no observations, so they keep caption-coverage wording and must not claim physical counts.
- The 30-question suite is a regression test, not a public benchmark. CapMem/EgoSchema/Ego4D would test questions we did not design.
- The VLM can hallucinate detail; captions need spot checks. Evidence stills and timestamped captions make the claim auditable.

15 References and papers used

The following works shaped the design. Citation numbers appear in the text where the idea is used.

References

[1] R4DSG. *Relative 4D scene-graph memory for egocentric video.* ACM MM 2026. arXiv:2608.11017. Anchor-relative, video-free scene memory; validates compact queryable entries.

[2] VL-KnG. *Persistent spatio-temporal knowledge graph from monocular egocentric video.* 2025/2026. arXiv:2510.01483. Closest architectural cousin: entities + events + time, query latency decoupled from video length.

[3] EgoGraph. *Training-free temporal knowledge graph for egocentric memory.* 2026. arXiv:2602.23709. Egocentric schema of people, objects, locations and events; temporal modelling across entities.

[4] CapMem. *Episodic memory video caption QA.* EMNLP 2026. arXiv:2609.17688. Captions-only QA beats direct VideoQA on long videos for most models; motivates the 30 s caption ledger and the caption-window study.

[5] EgoCITE. *Dense visual captions + speech transcripts as memory entries.* 2026. arXiv:2608.12627v2. Time is a first-class retrieval key; motivates timestamped windows and temporal filters.

[6] MERIT. *Query-agnostic memory build; episodic multi-key representation.* ECCV 2026. arXiv:2608.07663. Build high-recall memory before knowing the question; compose/expand only at query time.

[7] FocusGraph. *Scene-graph LLM selector + sparse keyframes.* 2026. arXiv:2603.04349. Text-first, pixels-second routing; supports the evidence-on-demand ladder.

[8] VideoMem. *Adaptive global memory buffer for ultra-long video.* 2025/2026. arXiv:2512.04540. Compression and retention policy inspiration.

[9] EM-LLM. *Episodic events from Bayesian surprise + graph boundary refinement.* ICLR 2025. ICLR proceedings. Event segmentation and similarity + temporal-contiguity retrieval.

[10] LightMem. *Sensory → short-term → long-term memory; offline consolidation.* ICLR 2026. arXiv:2510.18866. Tiering and the offline/online split; ingest is our offline phase.

[11] HippoRAG. *Knowledge graph + Personalized PageRank for long-term memory.* NeurIPS 2024. arXiv:2405.14831. Graph retrieval as a cheap alternative to iterative VLM calls.

[12] EgoMemory / EgoRetriever. *Personal memory bank + reflective CoT; training-free retrieval over long egocentric video.* ACL Findings 2026. ACL anthology. Personalisation and intent inference over a memory bank.

[13] Continuous Semantic Caching. *Principled semantic cache reuse under uncertainty.* 2026. CMU LIONS. The answer cache is a memory tier; paraphrases can reuse grounded answers.

[14] Titans. *Neural long-term memory that learns at test time.* 2024/2025. arXiv:2501.00663. North star: memory in weights, not only a database.

[15] MemoryLLM. *Fixed-size latent memory pool inside a transformer.* ICML 2024. arXiv:2402.04624. Alternative route to model-native episode memory; not on the demo critical path.

[16] Caption-once, Frames-on-Demand. *Cache captions once; fetch frames only when confidence is low.* EMNLP 2026. ACL anthology. Exactly the query-time cost model: text first, bounded pixels on demand.

Appendix: commands

```
# Start the merged zine UI (main UI + VFM memory)
cd /home/g1/projects/chronicle-vfm
./chronicle.sh start            # this worktree defaults to :8082
./chronicle.sh status
./chronicle.sh stop

# Full verification
./chronicle.sh test

# Compile a memory directly (no UI)
python prototype/compile.py --source-db data/real/cook/chronicle.db --out /tmp/cook.db
python prototype/attach_ledger.py --vault /tmp/cook.db --ledger /tmp/cook_ledger_30.json
python prototype/query.py --db /tmp/cook.db "how many bowls?"
python prototype/report.py --db /tmp/cook.db --out /tmp/cook.html --data-root /tmp

# 2k / 4k / 8k pack ceilings
python prototype/memory_pack.py --db /tmp/cook.db --list-budgets
```